

# 汇丰科技AI Coding工程实践

## 从AI Coding 到 AI Soft Engineering

李渭宁

汇丰科技-内部开源负责人

**AiCon**  
全球人工智能开发与应用大会



## 李渭宁

汇丰科技，内部开源技术委员会主席

金融科技

程序员

汇丰AI大使

18年金融科技从业经验

职位包含产品经理, 软件开发, 测试, 运维, 架构师, 项目经理, 交付经理, 服务过汇丰科技, 广汽汽车金融, 小鹏融资租赁, 恒生银行

# 极客邦科技 2026 年会议规划

促进软件开发及相关领域知识与创新的传播



参会咨询

InfoQ 技术大会

**QCon** 上海站  
全球软件开发大会

大模型正在重新定义软件  
Large Language Model Is Redefining  
The Software

10月22-24日 / 上海建工浦江皇冠假日酒店

· AI Agent

· Vibe Coding

· 智能可观测

· 推理基建

· 模型攻防

· AI x 创造力

InfoQ 技术大会

**AiCon** 北京站  
全球人工智能开发与应用大会

探索 AI 应用边界  
Explore the limits of AI applications

12月18-19日 / 北京

· 大模型架构创新

· 多模态 AI 产业融合

· 具身智能

· AI for Science

· 大模型安全

# 汇丰科技植根中国，服务全球



广州  
汇丰科技中国最早及最大的科技中心

西安  
第二大科技中心

上海  
技术服务团队

深圳  
创新孵化团队



# 目录

01 企业落地 AI Coding 的挑战

02 我们的应对策略

03 案例和经验分享

04 回顾和总结

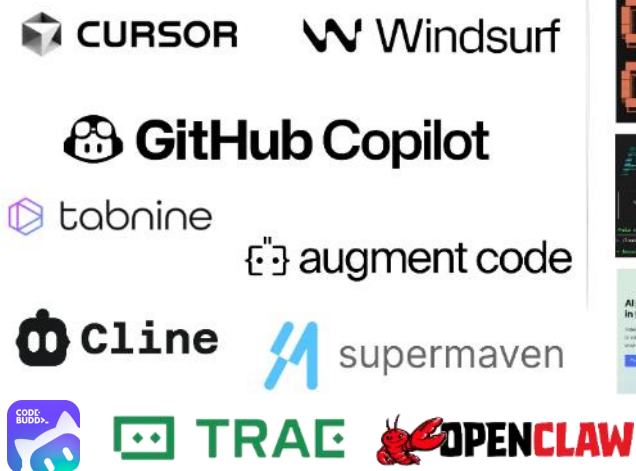
01

# 企业落地 AI Coding 的挑战

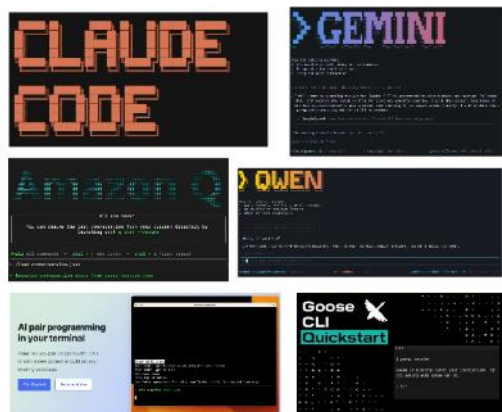
# AI Coding工具日新月异

AI coding 工具  
各领风骚一个月

## AI Code Editors and Extensions



## CLI AI Coding Agents



90%个人感知到效率提升

# DORA

Google Cloud

2025

AI's primary role in software development is that of an amplifier

AI in software development has become virtually ubiquitous:



### 01. Adoption

**90%** of technology professionals report using AI at work. Suggesting that AI use in software development has now become the standard.

### 02. Productivity

**>80%** of technology professionals report a perception that AI has increased their productivity.

### 03. Reliance

Technology professionals are heavily reliant on AI tools at work. **65%** report relying on AI at least a "moderate amount".

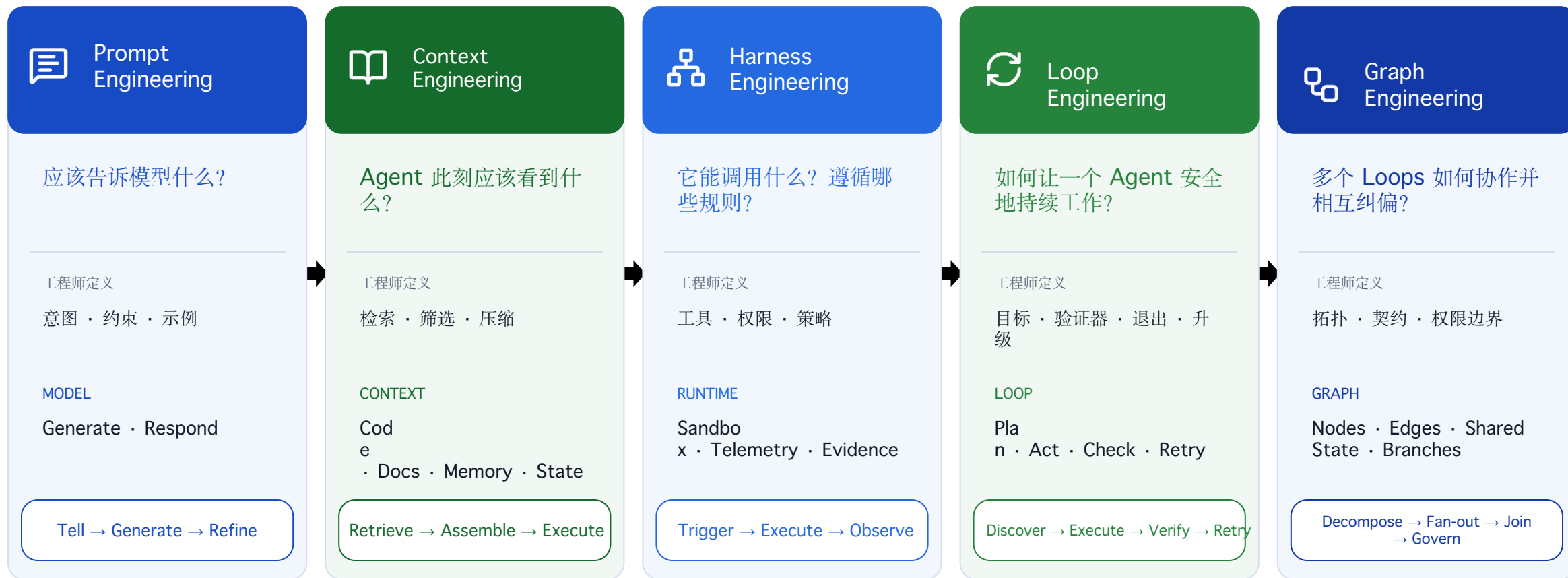
### 04. Reflexive use

**60%** of AI users surveyed use AI "about half the time or more" when encountering a problem to solve or task to complete, suggesting that AI has become a frequent part of the development process.

### 05. Code quality

**59%** of survey respondents also observe that AI has positively impacted their code quality.

# AI Coding方法论推陈出新



PDCA CONTROL LOOP



Plan



Do



Check

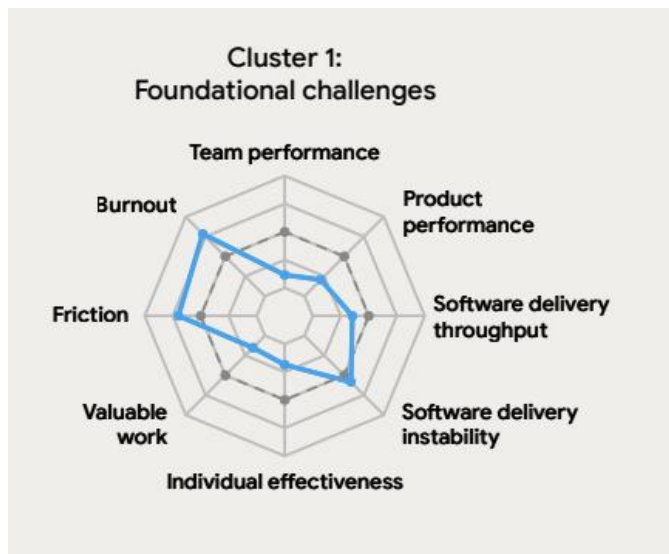


Act

同一套控制循环, 工程范围从单条指令扩展到可治理的多 Loop 网络

# 落地效果千差万别

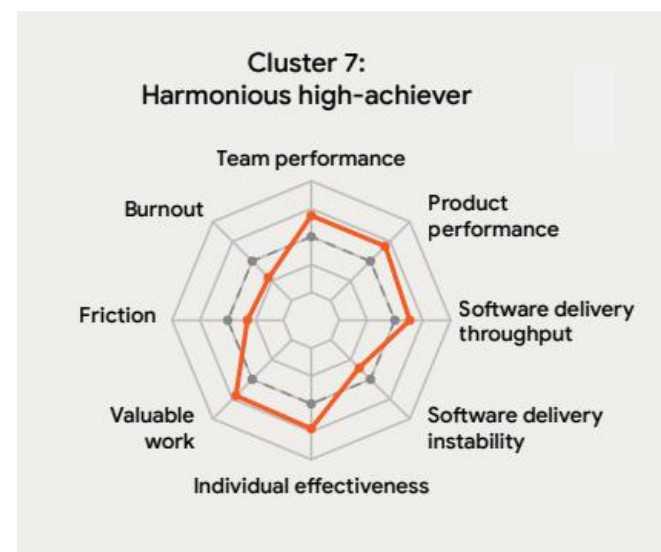
AI Coding 工具像放大器：强者更强，弱者更快暴露问题



## Bad teams

Higher technical debt and weaker controls

VS



## Good teams

Strong CI/CD, testing, observability, and disciplined ways of working

Refer to 2025 DORA report

# ■ 企业级 AI Coding 规模化落地的五项挑战

真正的规模化，不只是让个人写得更快，而是让效率、上下文、质量、安全和复用同时进入工程体系。

01



PRIMARY OUTCOME

## AI Software Engineering Efficiency

从个人提效，走向 End-to-End 研发效能提升

关键问题

AI 是否缩短了从需求到上线的完整链路？

LTTD · DORA · Release Frequency

Measure team outcomes, not lines of code

02 Enterprise Context



AI 是否理解企业技术约束？

Architecture · APIs · Standards · Internal Tools

Agent-ready Context

03 Quality Assurance



AI 加速后，Quality Gates 依然可靠？

Code Review · Testing · Scanning · Validation

Verified Evidence

04 Security & Governance



Guardrails 是否前置、可审计、可追溯？

Isolation · Identity · Data · Network · Policy

Governed Execution

05 Engineer Experience



工程师是否愿意用、会用、持续使用？

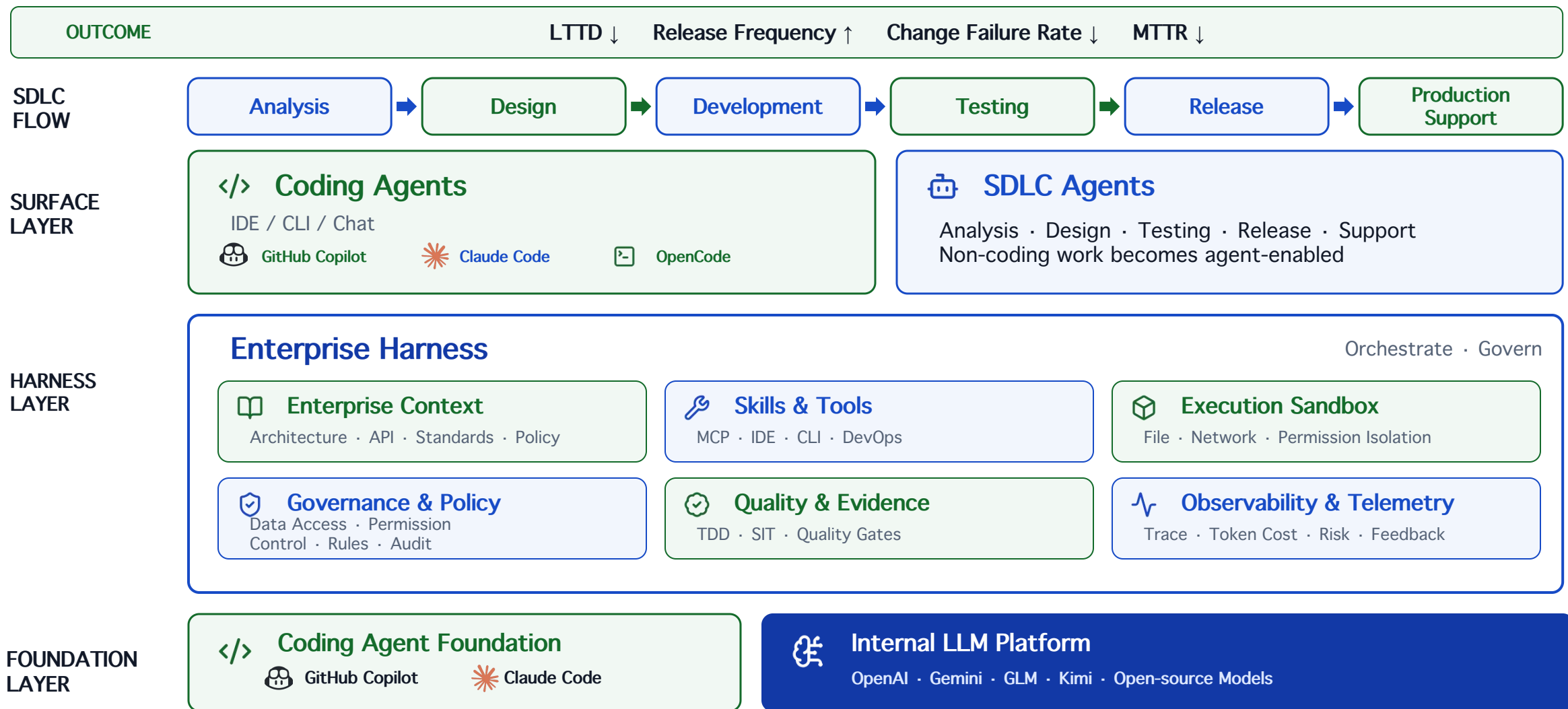
Workflow Fit · Ease of Use · Team Adoption

Sustainable Adoption

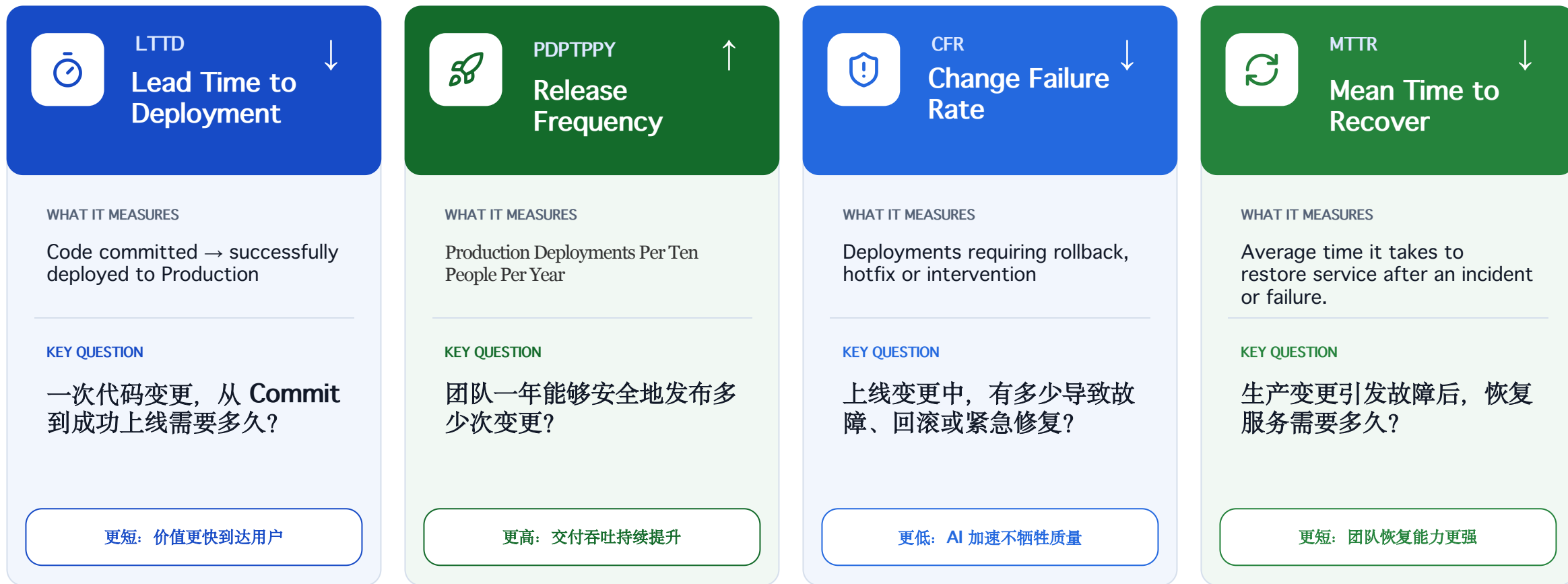
02

## 应对策略

# AI Software Engineering Overview



# 度量指标



AI CODING OUTCOME

LTTD ↓

Release Frequency ↑

Change Failure Rate ↓

Recovery Time ↓

# 企业内部上下文

## AGENT-READY CONTEXT

让 Agent 获取正确、受控、实时且可追溯的企业上下文



### 01 CODEBASE

#### OpenSpec

Distill implementation  
into reusable  
specifications

Specs · APIs · Constraints



### 02 ENTERPRISE KNOWLEDGE

#### Knowledge Service

Confluence · Jira · RAG  
as a Service

Relevant · Current · Traceable



### 03 INTERNAL TOOLS

#### MCP & Skills

Connect DevOps  
pipelines and internal  
services and Cyber Tools

Tools · Workflow · Automation

# AI Coding 加速之后, Quality Gates 必须更深入

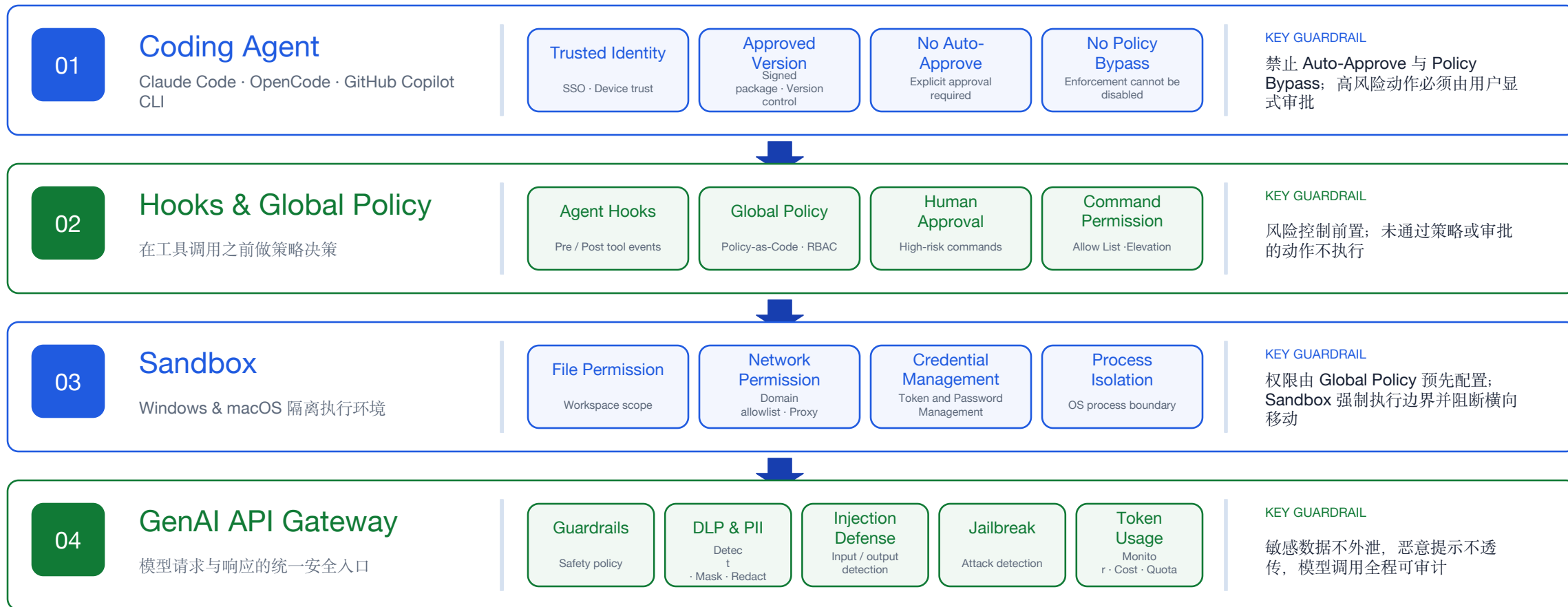


FASTER CODING  $\neq$  LOWER STANDARDS

质量标准不变; 验证更早、更深、更自动化, Evidence 贯穿整个 SDLC

# Coding Agent 安全管控：四层纵深防御

每一层承担不同控制职责：身份可信、策略前置、执行隔离、模型访问防护



CROSS-LAYER AUDIT & OBSERVABILITY

Default Deny

Least Privilege

Human-in-the-Loop

Traceable & Auditable

# Observability - Token 的使用

多数是敏捷小需求、快速迭代；只有 1-2% 在系统重构中，用 Token 将庞大代码库转化为 Spec。



# ■ 如何让工程师驾驭AI Coding?

工程师利用ETC掌握一门技术



# 落地方法

## E ffective Harness

高效驾驭

让 Agent 在企业边界内稳定运行

KEY CAPABILITIES

- Enterprise Context & Policy
- Quality & Evidence
- Sandbox & Observability

## T ooling

顺手工具

让 AI 进入开发者每天使用的工作流

KEY CAPABILITIES

- IDE / CLI / Chat
- Coding & SDLC Agents
- Low-friction Adoption

## C ommunity

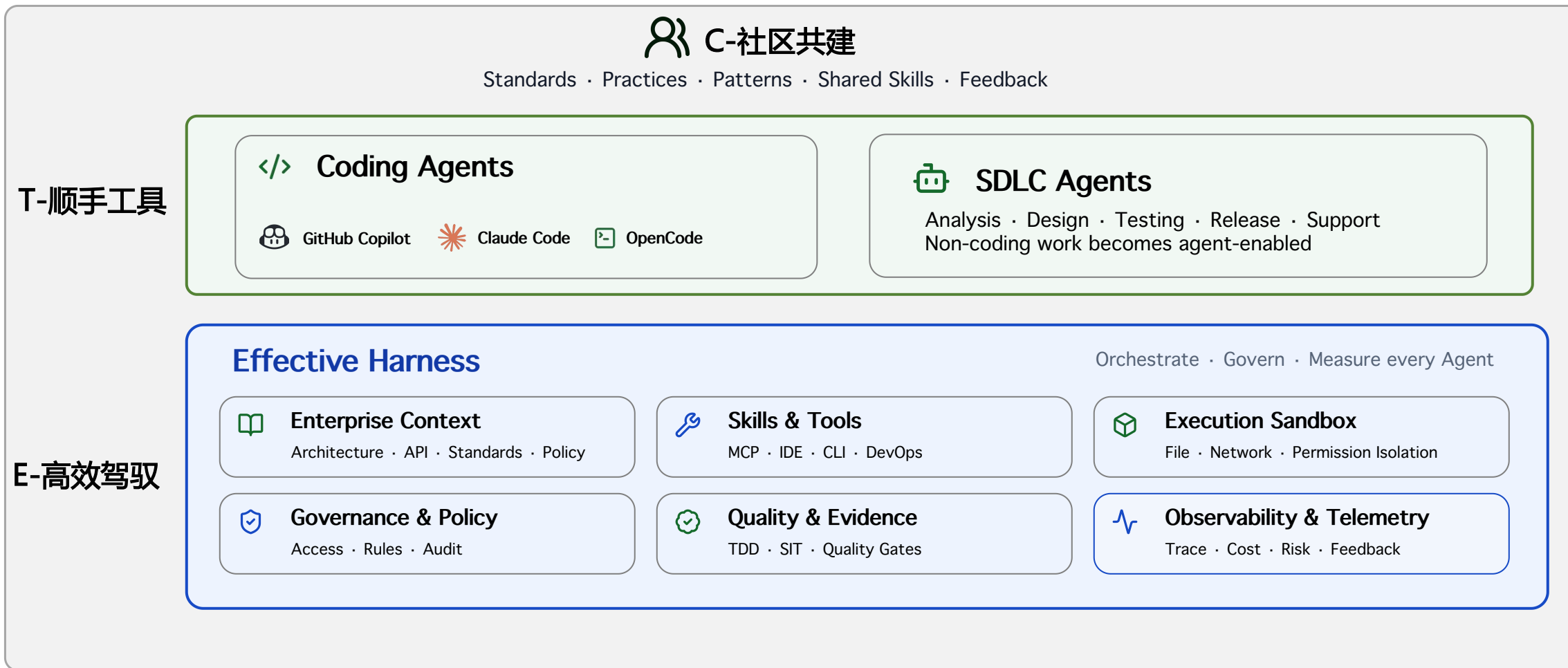
社区共建

让个人经验沉淀为可复用的组织能力

KEY CAPABILITIES

- Practices & Patterns
- Prompts & Skills
- Feedback & Champions

# ETC 共同构成企业级 AI Software Engineering



03

## 案例和经验分享

# 案例和经验分享

## C-社区共建

Standards · Practices · Patterns · Shared Skills · Feedback

### 端到端交付



DELIVERY PATTERNS

#### AI SDLC 开发方式

按场景选择 SDD、Multi-Agent 或 AI Assistance



CAPABILITY EVOLUTION

#### 工具与 Skills

持续涌现、评估、标准化与复用



EFFICIENCY

#### Token 优化

关注 Cost per Verified Outcome, 而非消耗率

### 场景分享



REQUIREMENT

#### 人和 AI 读懂同一份需求

用结构化 Spec、验收标准与边界条件消除歧义



CODE UNDERSTANDING

#### 先看懂 Program Logic

建立调用链、依赖与影响范围, 再开始修改



CODE REVIEW

#### 生成越快, Review 越重要

同时验证业务需求、内部技术规范、安全标准与 End-to-End Evidence



TDD

#### 保证 Coding 质量

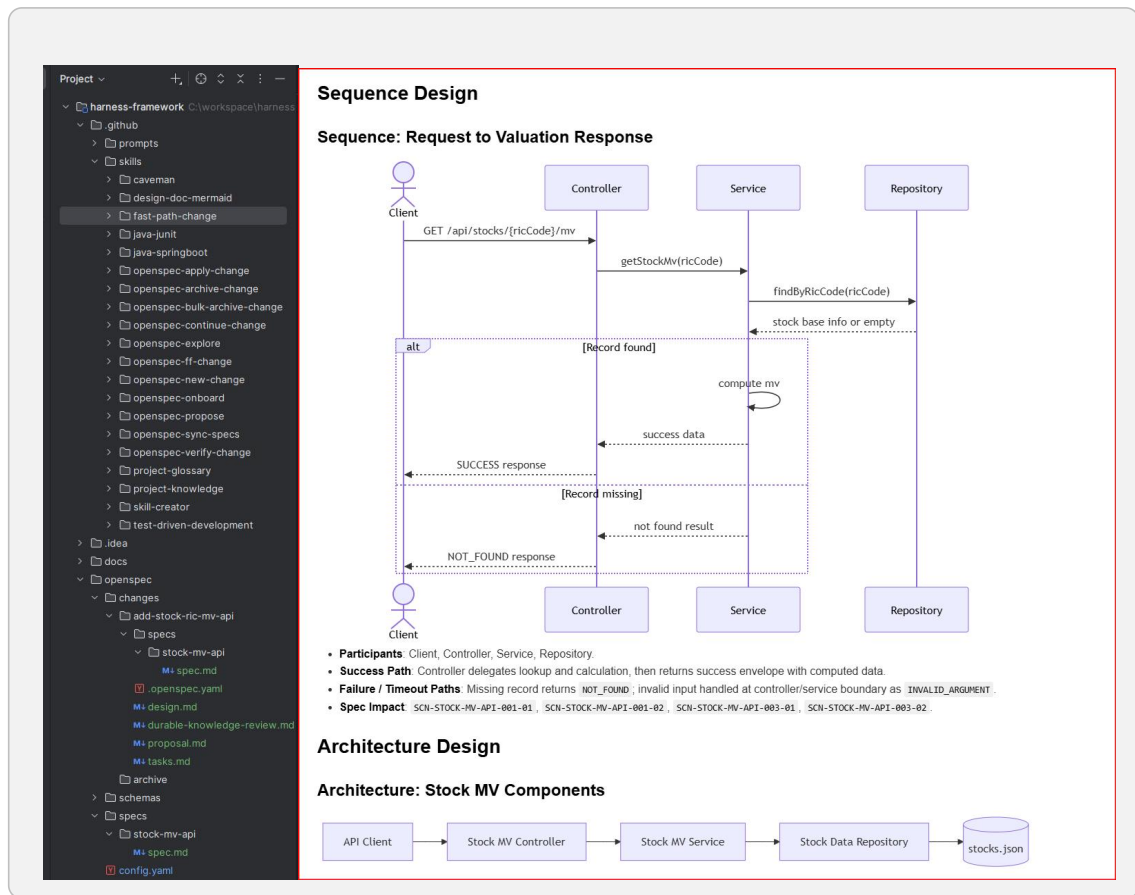
用 Simple、Boundary 与 Exception Cases 验证行为

# AI Coding 的场景矩阵



# SDD (Specification-Driven Development) 规范驱动开发

先把意图变成可执行规范，再让 Agent 在 Rules、Tests 与 Tooling 的约束下完成交付



**DUAL CONTROL**

**Rules** 定义逻辑边界，**TDD** 提供可执行质量契约

Specification constrains intent · Tests verify behavior · Agents automate execution



# TDD - AI 时代不可替代的“质量护栏”

## Simple Testing Can Prevent Most Critical Failures

An Analysis of Production Failures in Distributed Data-intensive Systems

Ding Yuan, Yu Luo, Xin Zhuang, Guilherme Renna Rodrigues, Xu Zhao,

Yongli Zhang, Pokey U. Jee, Michele S. ...

University of Toronto

A majority of the production failures (77%) can be reproduced by a unit test.

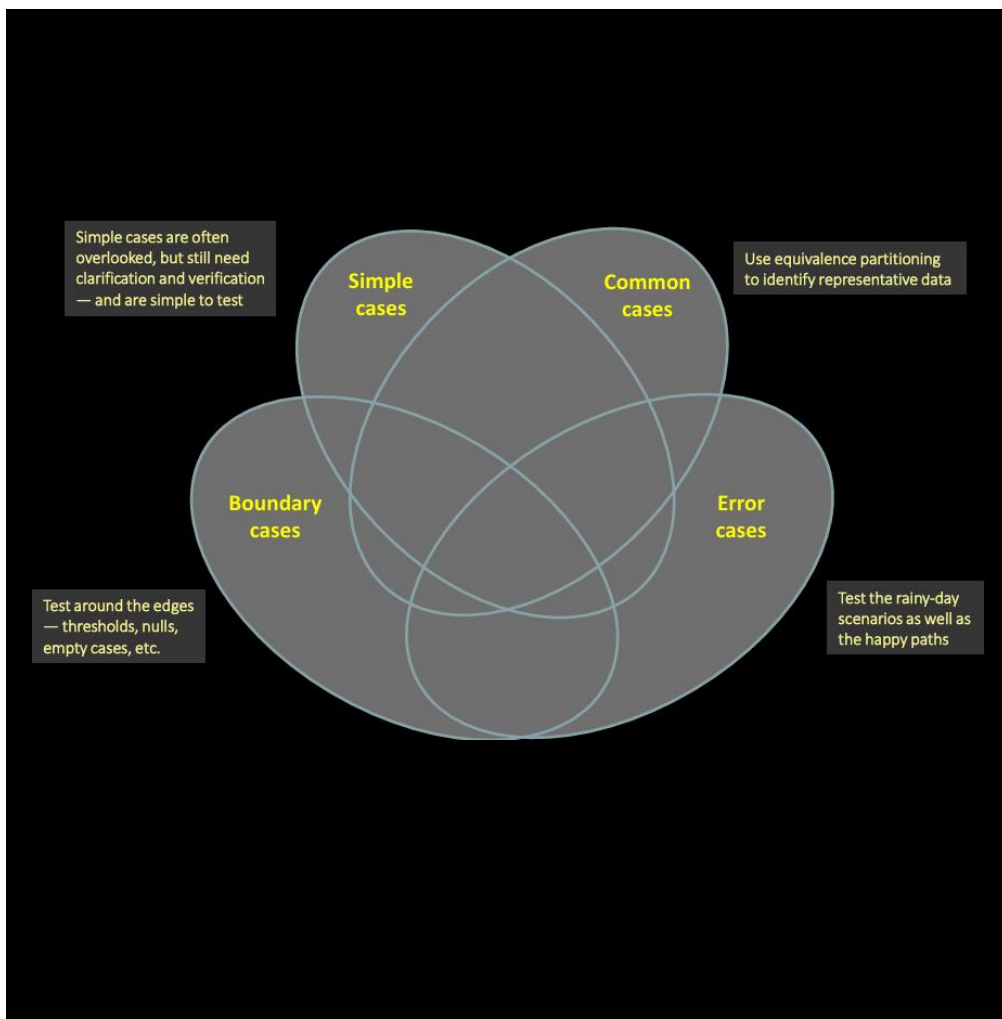
### Abstract

Large, production-quality distributed systems still fail periodically, and do so sometimes catastrophically, where most or all users experience an outage or data loss. We present the result of a comprehensive study investigating 198 randomly selected, user-reported failures that occurred on Cassandra, HBase, Hadoop Distributed File System (HDFS), Hadoop MapReduce, and Redis, with the goal of understanding how one or multiple faults eventually evolve into a user-visible failure. We found that from a testing point of view, almost all failures require only 3 or fewer nodes to reproduce, which is good news considering that these services typically run on a very large number of nodes. However, multiple inputs are needed to trigger the failures with the order between

raises the question of why these systems still experience failures and what can be done to increase their resiliency. To help answer these questions, we studied 198 randomly sampled, user-reported failures of five data-intensive distributed systems that were designed to tolerate component failures and are widely used in production environments. The specific systems we considered were Cassandra, HBase, Hadoop Distributed File System (HDFS), Hadoop MapReduce, and Redis.

Our goal is to better understand the specific failure manifestation sequences that occurred in these systems in order to identify opportunities for improving their availability and resiliency. Specifically, we want to better understand how one or multiple errors<sup>1</sup> evolve into component failures and how some of them eventually

# 如何写好写全TDD案例



## Given

... a particular set-up and initial state...

## When

... a particular operation is performed...

## Then

... a particular outcome is expected

```
public class Queue_spec
{
    @Test
    public void A_new_queue_is_empty()
    {
        // When:
        Queue<Integer> empty = new Queue<>();
        // Then:
        assertEquals(0, empty.length());
    }
    @Test
    public void An_empty_queue_dequeues_no_values()
    {
        // Given:
        Queue<Integer> empty = new Queue<>();
        // When:
        var dequeued = empty.dequeue();
        // Then:
        assertTrue(dequeued.isEmpty());
    }
    @Test
    public void An_empty_queue_becomes_longer_when_value_enqueued()
    {
        // Given:
        Queue<String> queue = new Queue<>();
        // When:
        queue.enqueue("42");
        // Then:
        assertEquals(1, queue.length());
    }
    @Test
    public void A_non_empty_queue_becomes_longer_when_value_enqueued()
    {
        // Given:
        Queue<String> queue = new Queue<>();
        queue.enqueue("Hello");
        // When:
        queue.enqueue("World");
        // Then:
        assertEquals(2, queue.length());
    }
    @Test
    public void A_non_empty_queue_dequeues_values_in_order_enqueued()
    {
        // Given:
        Queue<String> queue = new Queue<>();
        queue.enqueue("Hello");
        queue.enqueue("World");
        // When:
        String[] dequeued = {queue.dequeue().get(), queue.dequeue().get()};
        // Then:
        assertEquals(0, queue.length());
        assertEquals("Hello", dequeued[0]);
        assertEquals("World", dequeued[1]);
    }
}
```

# Multi-Agent 多智能体协同交付模式

不同 Agent 按角色分工，通过 Shared Context、Review Gates 与 Graph Orchestration 完成端到端任务



## CORE ADVANTAGE

模块化解耦上下文，  
通过对偶 **Review** 阻断错误级联

- Role-based Context: 每个 Agent 只加载所需上下文
- Review Gates: 关键交付物由独立 Agent 复核
- Parallel Execution: 可并行任务缩短交付周期

老A, 小莫已为您顺利完成任务 **PCTECH-2007** 的全流程 SDLC 调度与交付!

### ■ 任务交付概览

- Jira 编号: PCTECH-2007
- 目标组件: rayleigh-pxr
- 特性分支: feature/PCTECH-2007-rayleigh-pxr-foss-vulnerability-remediation-dependency-upgrades
- Git Commit SHA: bd33904
- 远程推送状态: 成功 (已推送至 HSBC Github Enterprise 远程分支)
- 工作站归档路径: op-workstations/archive/PCTECH-2007/

### ⚙️ 流水线各阶段执行结果

| 阶段           | 负责 Agent | 执行状态       | 产出物 / 结果摘要                                                                      |
|--------------|----------|------------|---------------------------------------------------------------------------------|
| 1. 需求分析与设计   | e1and    | COMPLETE D | 生成 TOD 规范文件 PCTECH-2007-requirement.md, 明确 7 项 Foss 依赖/插件升级规则及回归范围              |
| 2. 核心编码与单元测试 | aeg15    | COMPLETE D | 完成 pom.xml 依赖升级, 31 个单元测试 100% 通过, 全局代码覆盖率 95.78%                               |
| 3. 微服务集成测试   | hugo     | COMPLETE D | 评级 Grade 2, 构建并联动 worldgovt-eureka, chopper-pxr, thousand-sunny-pxr, 3 个集成断言全通过 |
| 4. 文档与一致性审计  | r1cky    | COMPLETE D | 审计结果 PASSED, 刷新并确保 README.md 与 AGENTS.md 100% 英文精准无误                            |
| 5. 代码提交与远端推送 | th0mas   | COMPLETE D | 符合 EQD-OP 规范的前端提交 (PCTECH-2007: ...) 并成功推送至远程 origin                            |
| 6. 工作站归档     | th0mas   | COMPLETE D | 成功将任务目录整体移至 op-workstations/archive/PCTECH-2007/                                |

# Agent 工作的核心：PDCA 持续闭环



HARNESS 的价值：让每一次循环都有边界、有证据、可观测、可恢复

# 案例和经验分享

## C-社区共建

Standards · Practices · Patterns · Shared Skills · Feedback

### 端到端交付



DELIVERY PATTERNS

#### AI SDLC 开发方式

按场景选择 SDD、Multi-Agent 或 AI Assistance



CAPABILITY EVOLUTION

#### 工具与 Skills

持续涌现、评估、标准化与复用



EFFICIENCY

#### Token 优化

关注 Cost per Verified Outcome, 而非消耗率

### 场景分享



REQUIREMENT

#### 人和 AI 读懂同一份需求

用结构化 Spec、验收标准与边界条件消除歧义



CODE UNDERSTANDING

#### 先看懂 Program Logic

建立调用链、依赖与影响范围, 再开始修改



CODE REVIEW

#### 生成越快, Review 越重要

同时验证业务需求、内部技术规范、安全标准与 End-to-End Evidence



TDD

#### 保证 Coding 质量

用 Simple、Boundary 与 Exception Cases 验证行为

# 工具 4 维评估矩阵

## 企业内部开源组件智能评测平台



# 评估报告

## 批量评估

Round 1 Shortlisted Projects

All Projects

| #  | Project                                   | Score | Potential | Completeness | Code Quality | Community | Reusability | Knowledge | Report                      |
|----|-------------------------------------------|-------|-----------|--------------|--------------|-----------|-------------|-----------|-----------------------------|
| 2  | Atlas Phoenix Lens                        | 7.50  | 8.5       | 8.0          | 8.0          | 7.5       | 5.5         | 7.0       | <a href="#">Open Report</a> |
| 3  | Atlas Engineering Delivery Hub            | 7.33  | 7.5       | 8.0          | 8.0          | 8.0       | 5.0         | 6.5       | <a href="#">Open Report</a> |
| 4  | Altas Engineering Delivery Hub - Build    | 6.83  | 8.0       | 7.0          | 7.2          | 5.5       | 6.0         | 7.8       | <a href="#">Open Report</a> |
| 7  | Altas Engineering Delivery Hub - Planning | 7.38  | 8.0       | 8.0          | 8.0          | 7.5       | 5.5         | 6.5       | <a href="#">Open Report</a> |
| 8  | HSBC Org Explorer                         | 4.87  | 6.8       | 5.0          | 5.8          | 3.0       | 3.8         | 4.8       | <a href="#">Open Report</a> |
| 9  | Go Nexus Checker                          | 5.44  | 6.5       | 5.5          | 7.2          | 3.2       | 4.0         | 6.0       | <a href="#">Open Report</a> |
| 10 | Agentic PR Review                         | 5.36  | 7.2       | 5.2          | 6.8          | 3.5       | 4.0         | 5.0       | <a href="#">Open Report</a> |
| 11 | Go Skill Manager                          | 6.42  | 6.2       | 7.6          | 7.0          | 7.8       | 4.0         | 4.4       | <a href="#">Open Report</a> |
| 12 | Rotamaster alerting                       | 6.18  | 7.8       | 7.6          | 7.0          | 3.6       | 6.4         | 4.4       | <a href="#">Open Report</a> |
| 14 | RPGLE Mock Debugger (VSCode Plugin)       | 6.92  | 7.8       | 7.1          | 7.6          | 7.0       | 5.4         | 5.8       | <a href="#">Report</a>      |
| 15 | InfraOps CostPilot                        | 6.25  | 7.8       | 7.6          | 7.0          | 4.6       | 5.0         | 5.2       | <a href="#">Open Report</a> |
| 16 | AI Code Review                            | 7.39  | 8.3       | 8.0          | 7.4          | 6.6       | 7.5         | 6.5       | <a href="#">Open Report</a> |
| 17 | Crash Assistant                           | 6.62  | 6.2       | 7.6          | 7.0          | 7.8       | 4.0         | 6.4       | <a href="#">Open Report</a> |
| 18 | Enhanced MCPs                             | 5.96  | 8.0       | 6.6          | 6.2          | 4.0       | 6.0         | 5.2       | <a href="#">Open Report</a> |
| 19 | Nebula                                    | 3.75  | 7.8       | 3.6          | 2.8          | 1.0       | 4.0         | 5.4       | <a href="#">Open Report</a> |
| 20 | System Mesh                               | 2.53  | 6.2       | 2.4          | 1.2          | 1.0       | 3.2         | 2.6       | <a href="#">Open Report</a> |
| 21 | AI Psychological Coach Agent              | 2.92  | 7.8       | 2.4          | 1.2          | 1.0       | 4.2         | 2.6       | <a href="#">Open Report</a> |
| 22 | Digital Avatar Brand Ambassador ?"Linky"  | 3.00  | 7.8       | 2.4          | 1.2          | 1.0       | 4.2         | 3.4       | <a href="#">Open Report</a> |
| 23 | IP-Calculator                             | 4.34  | 4.0       | 4.8          | 6.2          | 2.6       | 3.4         | 4.4       | <a href="#">Open Report</a> |
| 24 | AMH AI Data Lineage                       | 6.12  | 6.2       | 6.2          | 7.0          | 7.8       | 3.4         | 4.4       | <a href="#">Open Report</a> |
| 25 | SRE Incident Console                      | 2.68  | 7.2       | 2.4          | 1.2          | 1.0       | 3.2         | 2.6       | <a href="#">Open Report</a> |
| 26 | D365 Odata operation processor            | 6.18  | 6.2       | 7.6          | 7.0          | 6.6       | 4.0         | 4.4       | <a href="#">Open Report</a> |

## 产品详细报告

OPENCOLLABORATION 2026

### Atlas Phoenix Lens

Decision-support report generated from project submission evidence and live ALM GitHub repository metrics. Scores are evidence-based and intended for committee review, not automatic approval.

This AI Evaluation Report is based on project registration details, repository statistics, and evaluation criteria. Its purpose is to provide objective evidence to support human decision-making. The final decision will consider additional factors and be announced by the OpenCollaboration\_Core\_C. The committee reserves the right to the final interpretation.

|                      |                           |
|----------------------|---------------------------|
| Final score          | 7.50 / 10                 |
| Evaluation generated | 2026-07-10 04:26:20PM HKT |
| AI Evaluation Model  | GitHub Copilot GPT-5.5    |

#### SCORE DASHBOARD Six-Dimension Scorecard

| Dimension                       | Score | Weight | Weighted | Evidence |
|---------------------------------|-------|--------|----------|----------|
| Potential Value                 | 8.5   | 15%    | 1.275    | Partial  |
| Completeness                    | 8.0   | 15%    | 1.200    | Partial  |
| Code Quality                    | 8.0   | 25%    | 2.000    | Partial  |
| InnerSource Community Activity  | 7.5   | 20%    | 1.500    | Partial  |
| Reusability                     | 5.5   | 15%    | 0.825    | Partial  |
| Knowledge Sharing and Promotion | 7.0   | 10%    | 0.700    | Partial  |



#### DIMENSION ANALYSIS

##### Potential Value

Submission describes a strong modernization discovery value story: recover RPG/CL/COBOL/DDS behavior, preserve source evidence, surface SME questions, and prepare trusted modernization inputs.

| Metric          | Score | Status  | Explanation                                                                                                               |
|-----------------|-------|---------|---------------------------------------------------------------------------------------------------------------------------|
| Problem Solving | 9     | Ready   | Submission targets RPG/CL/COBOL/DDS behavior recovery, ARCAD REF/AREF flow discovery, and modernization evidence capture. |
| Business Value  | 9     | Ready   | Evidence supports BRDs, gap analysis, target architecture planning, application retirement, and AI-native SDLC handoff.   |
| Project Roadmap | 7     | Partial | M3 Discovery placement in the Seven Mountains SDLC narrative gives direction, but explicit milestones are missing.        |

Summary: The business value is strong, but roadmap milestone evidence is missing.

#### DIMENSION ANALYSIS

##### Completeness

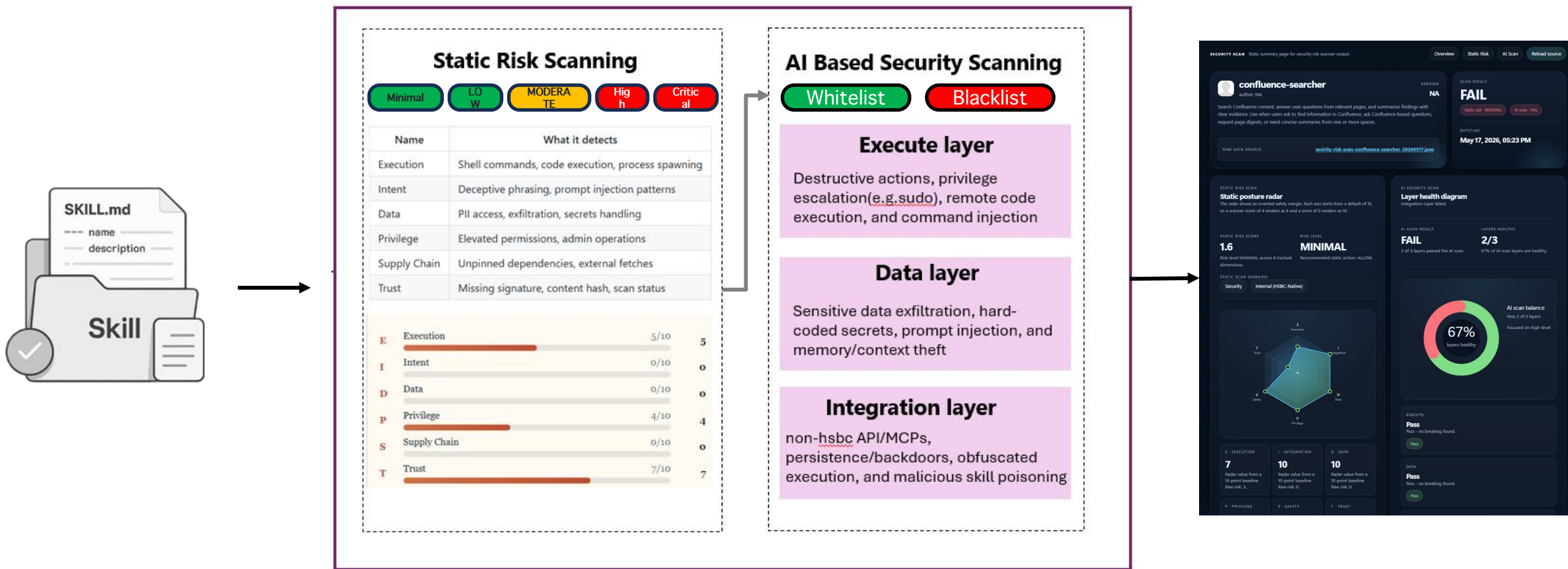
Evidence includes two repositories, README evidence, repository structure, and six contributor detail rows with HEET enrichment for numeric staff IDs.

| Metric            | Score | Status  | Explanation                                                                                                                   |
|-------------------|-------|---------|-------------------------------------------------------------------------------------------------------------------------------|
| Team Structure    | 8     | Ready   | GitHub contributors API returned 6 contributor detail rows across two repositories.                                           |
| Team Diversity    | 7     | Partial | HEET enrichment confirms contributors from HK TECH RBW - LENDING and AMH TECH AMH CIO OFFICE G2; service account remains N/A. |
| Product Readiness | 8     | Ready   | Submission describes program-flow discovery and source-code scanning capabilities.                                            |
| Documentation     | 9     | Ready   | README, quickstart, contributing, docs, schemas, scripts, and templates are available.                                        |
| DevOps Pipeline   | 6     | Partial | Repository structure includes github/scripts/tests, but workflow content was not fully inspected.                             |

Summary: Completeness is strong for repository and documentation evidence. Explicit roadmap, adoption fields, and expert validation remain missing.

# Agent Skill 质量安全评估

## Security and Risk Scanner



# 案例和经验分享

## C-社区共建

Standards · Practices · Patterns · Shared Skills · Feedback

### 端到端交付



DELIVERY PATTERNS

#### AI SDLC 开发方式

按场景选择 SDD、Multi-Agent 或 AI Assistance



CAPABILITY EVOLUTION

#### 工具与 Skills

持续涌现、评估、标准化与复用



EFFICIENCY

#### Token 优化

关注 Cost per Verified Outcome, 而非消耗率

### 场景分享



REQUIREMENT

#### 人和 AI 读懂同一份需求

用结构化 Spec、验收标准与边界条件消除歧义



CODE UNDERSTANDING

#### 先看懂 Program Logic

建立调用链、依赖与影响范围, 再开始修改



CODE REVIEW

#### 生成越快, Review 越重要

同时验证业务需求、内部技术规范、安全标准与 End-to-End Evidence

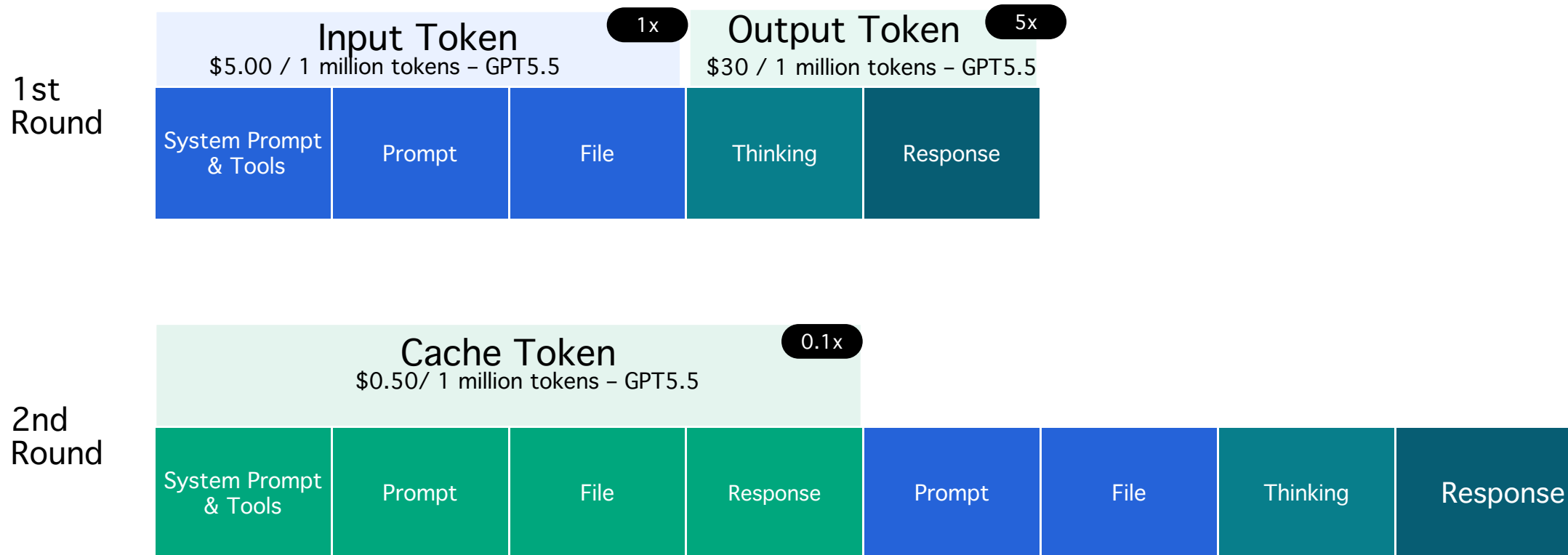


TDD

#### 保证 Coding 质量

用 Simple、Boundary 与 Exception Cases 验证行为

# Token如何计费?



# ■ 节省Token 的三板斧头



## Model-Choice

Dynamically match the optimal model based on task characteristics to balance cost (10% saving for auto-model)



## Tighten I/O

Use BLUF inputs and concise outputs. Use “Caveman Speaking” (Lite/Full/Ultra): simple, minimal prompts and instructions, short sentences, and few qualifiers—so the model is less likely to misread what you mean.



## Remove Noise

- Keep files small and focused
- Use clear names and structure
- Start new session when context gets long

# 节省Token10 技巧

**01 Turn ON Auto Model**  
Auto-selects the most cost-efficient model per task. Saves ~10% tokens.

**03 Caveman Speak**  
Leverage BLUF method to keep prompts short and concise

**05 Ask Mode > Agent Mode**  
Use Ask mode for general queries; reserve Agent mode for complex multi-step tasks.

**07 Search, Plan, Execute**  
This method separates the high-token thinking phase from the low-token execution phase

**09 Never Paste Large Files**  
Pasting large files wastes massive tokens. Use @file to reference instead.

**02 Control Your Output**  
output token price is **5x** more than standard input tokens, optimize output via [.github/copilot-instructions.md](https://github.com/copilot-instructions.md)

**04 Close Unused Editor Tabs**  
Unused open files may be auto-uploaded as context. Close to reduce noise.

**06 Start New Session**  
Start a new chat when context gets long. Old messages keep consuming tokens.

**08 Split Large Tasks**  
Break complex requests into smaller, focused prompts instead of one giant request.

**10 Scope Context with @file**  
Explicitly specify which files to reference. Never let Copilot guess scope.

Model-Choice

Tighten I/O

Remove Noise

# 案例和经验分享

## C-社区共建

Standards · Practices · Patterns · Shared Skills · Feedback

### 端到端交付



DELIVERY PATTERNS

#### AI SDLC 开发方式

按场景选择 SDD、Multi-Agent 或 AI Assistance



CAPABILITY EVOLUTION

#### 工具与 Skills

持续涌现、评估、标准化与复用



EFFICIENCY

#### Token 优化

关注 Cost per Verified Outcome, 而非消耗率

### 场景分享



REQUIREMENT

#### 人和 AI 读懂同一份需求

用结构化 Spec、验收标准与边界条件消除歧义



CODE UNDERSTANDING

#### 先看懂 Program Logic

建立调用链、依赖与影响范围, 再开始修改



TDD

#### 保证 Coding 质量

用 Simple、Boundary 与 Exception Cases 验证行为

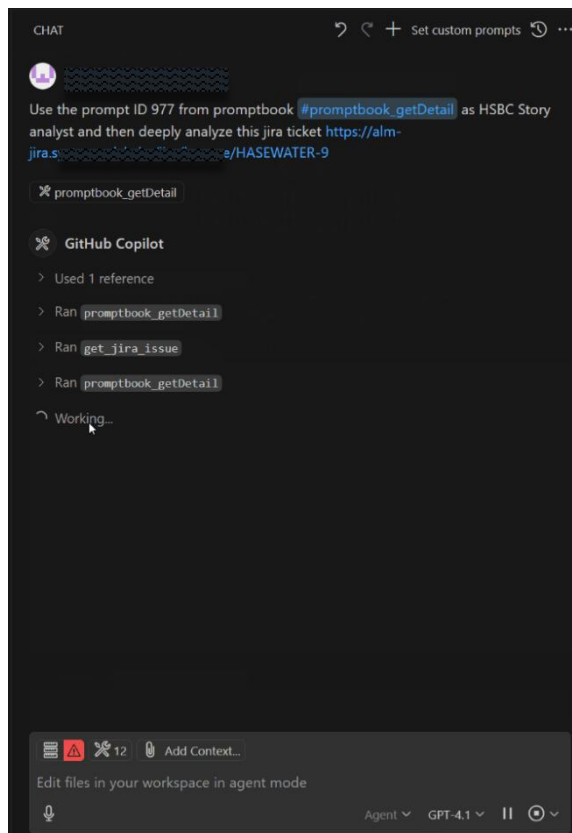


CODE REVIEW

#### 生成越快, Review 越重要

同时验证业务需求、内部技术规范、安全标准与 End-to-End Evidence

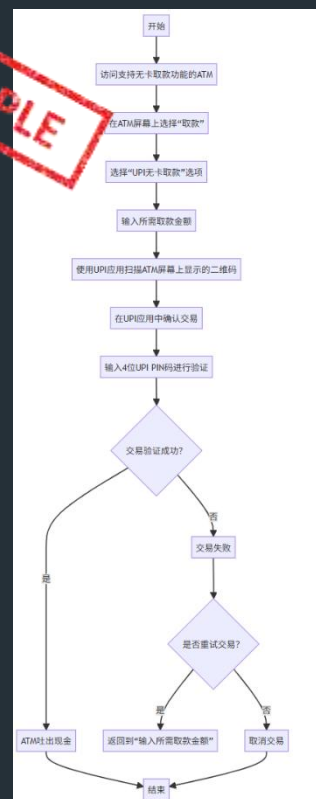
# 可视化用户需求



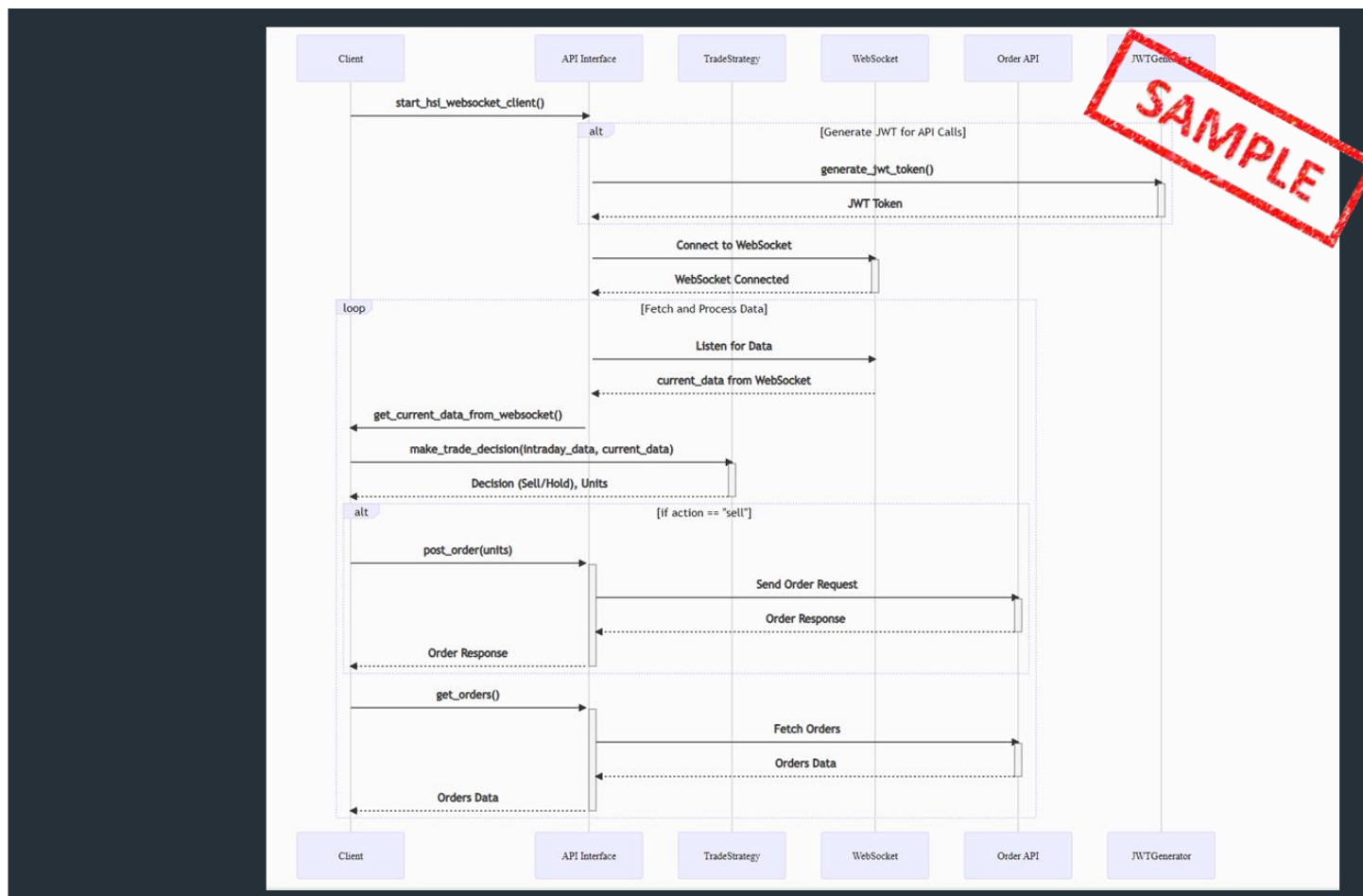
## Jira User Requirement

To withdraw money from an ATM using UPI, first visit an ATM that offers a cardless cash withdrawal facility. On the ATM screen, select the “Withdraw Cash” option followed by the “UPI Cardless Cash” option. Next, choose or enter the desired withdrawal amount carefully. Once the amount is selected, scan the QR code displayed on the ATM screen using your UPI app. After scanning the QR code, your UPI app will prompt you to confirm the transaction. Enter your 4-digit UPI PIN to validate the withdrawal. Once the transaction is successfully validated, the ATM will dispense the desired cash.

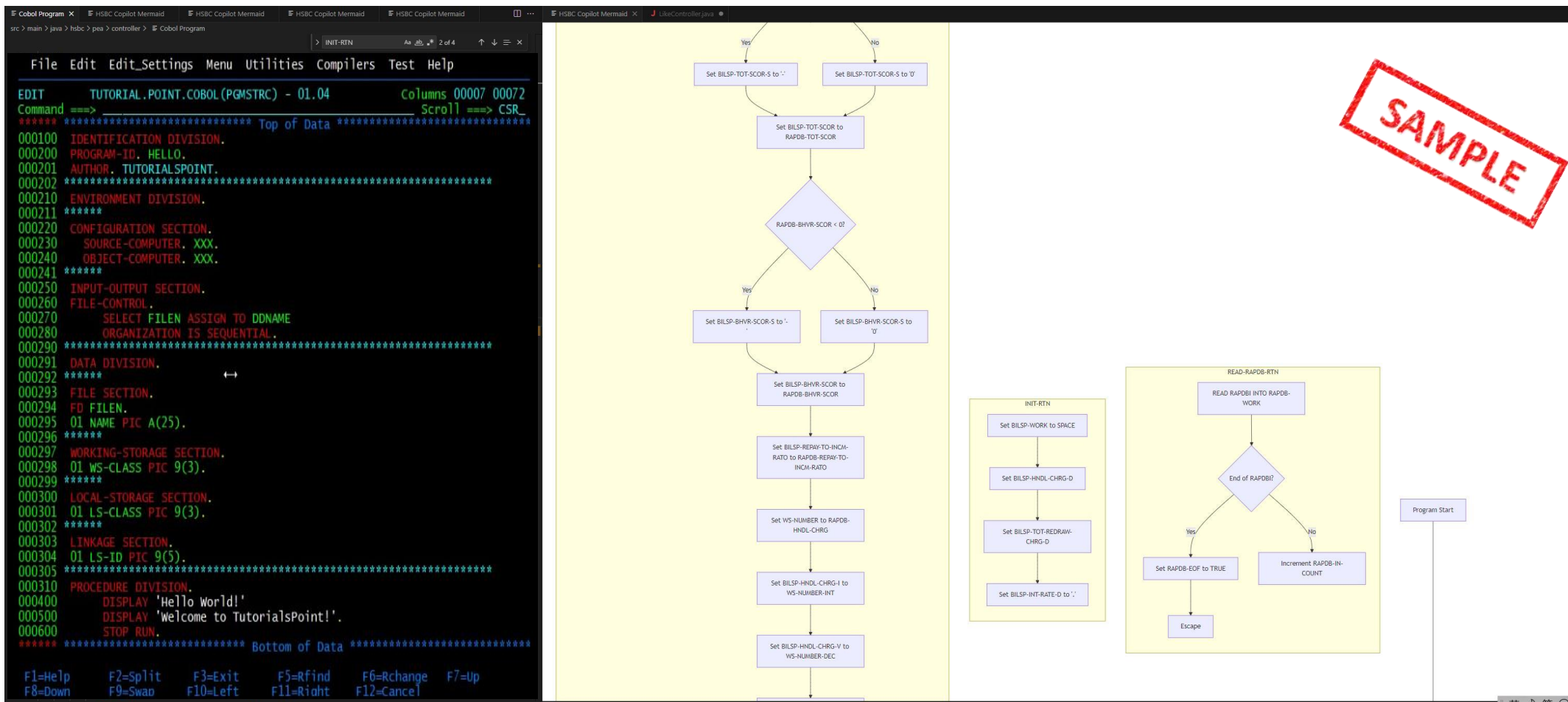
SAMPLE



# 程序逻辑可视化



# 程序逻辑可视化



# AI 驱动的高效代码评审机制

业务需求

非功能性需求

技术规范

安全规范

AI CODE REVIEWER: AI REVIEW CHAT

REVIEW SETTINGS  
Comment | GPT-5.4 mini | Chinese | Full

CREDENTIALS  
Configure tokens before entering the PR  
GitHub and JIRA credentials are configured and masked in SecretStorage.  
GitHub ready 1 optional token

Enter PR URL or Number...

Review

Post as PR Review Open diff in editor Clear chat

COPILOT MODEL  
GPT-5.4 mini  
gpt-5.4-mini | gpt-5.4-mini | max input 271,790 tokens

REVIEW LANGUAGE  
Chinese  
Choose the language for the generated review output.

OUTPUT MODE  
Full  
Compact posts a short reviewer summary. Custom lets you choose which sections appear in the draft before posting.

Output style: full

Review Result

Review Overview

| Topic             | Summary                                                                                                                                                                                                                                                                                                                                                                       |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Recommendation    | can merge.                                                                                                                                                                                                                                                                                                                                                                    |
| Main Reason       | The code change directly implements the intended fix (uses run date instead of system date) with low technical risk and no security issues. However, automated test coverage and end-to-end verification are missing and should be added as follow-ups to increase confidence; these are recommended but not blocking for merge if you accept the residual verification risk. |
| Requirement Focus | OPPMDS-57627 - 3RD-Pre-prod: 【30Dec2025】 【Data Compare Not Align】<br>Table_SSCUSTP_Filed_ZGMSD1                                                                                                                                                                                                                                                                               |
| Top Blocker       | None blocking for merge, but strongly recommend adding tests and confirming runDate format before promoting to pre-prod.                                                                                                                                                                                                                                                      |
| Top Finding 1     | High: None blocking for merge, but strongly recommend adding tests and confirming runDate format before promoting to pre-prod.                                                                                                                                                                                                                                                |
| Top Finding 2     | Watch area: Testing scored 60/100.                                                                                                                                                                                                                                                                                                                                            |
| Top Finding 3     | Requirement risk: verify acceptance criteria coverage against the linked JIRA ticket.                                                                                                                                                                                                                                                                                         |

Review Snapshot

| Area              | Signal  | Result       |
|-------------------|---------|--------------|
| Merge Decision    | 👍 Good  | CAN_MERGE    |
| Overall Score     | 👍 Good  | 82/100       |
| Testing           | 👎 Risk  | 60/100       |
| Requirement Match | 👎 Mixed | Needs review |
| Code Quality      | 👍 Good  | 80/100       |
| Correctness       | 👍 Good  | 85/100       |
| Performance       | 👍 Good  | 90/100       |
| Security          | 👍 Good  | 95/100       |

# 案例和经验分享

## C-社区共建

Standards · Practices · Patterns · Shared Skills · Feedback

### 端到端交付



DELIVERY PATTERNS

#### AI SDLC 开发方式

按场景选择 SDD、Multi-Agent 或 AI Assistance



CAPABILITY EVOLUTION

#### 工具与 Skills

持续涌现、评估、标准化与复用



EFFICIENCY

#### Token 优化

关注 Cost per Verified Outcome, 而非消耗率

### 场景分享



REQUIREMENT

#### 人和 AI 读懂同一份需求

用结构化 Spec、验收标准与边界条件消除歧义



CODE UNDERSTANDING

#### 先看懂 Program Logic

建立调用链、依赖与影响范围, 再开始修改



TDD

#### 保证 Coding 质量

用 Simple、Boundary 与 Exception Cases 验证行为

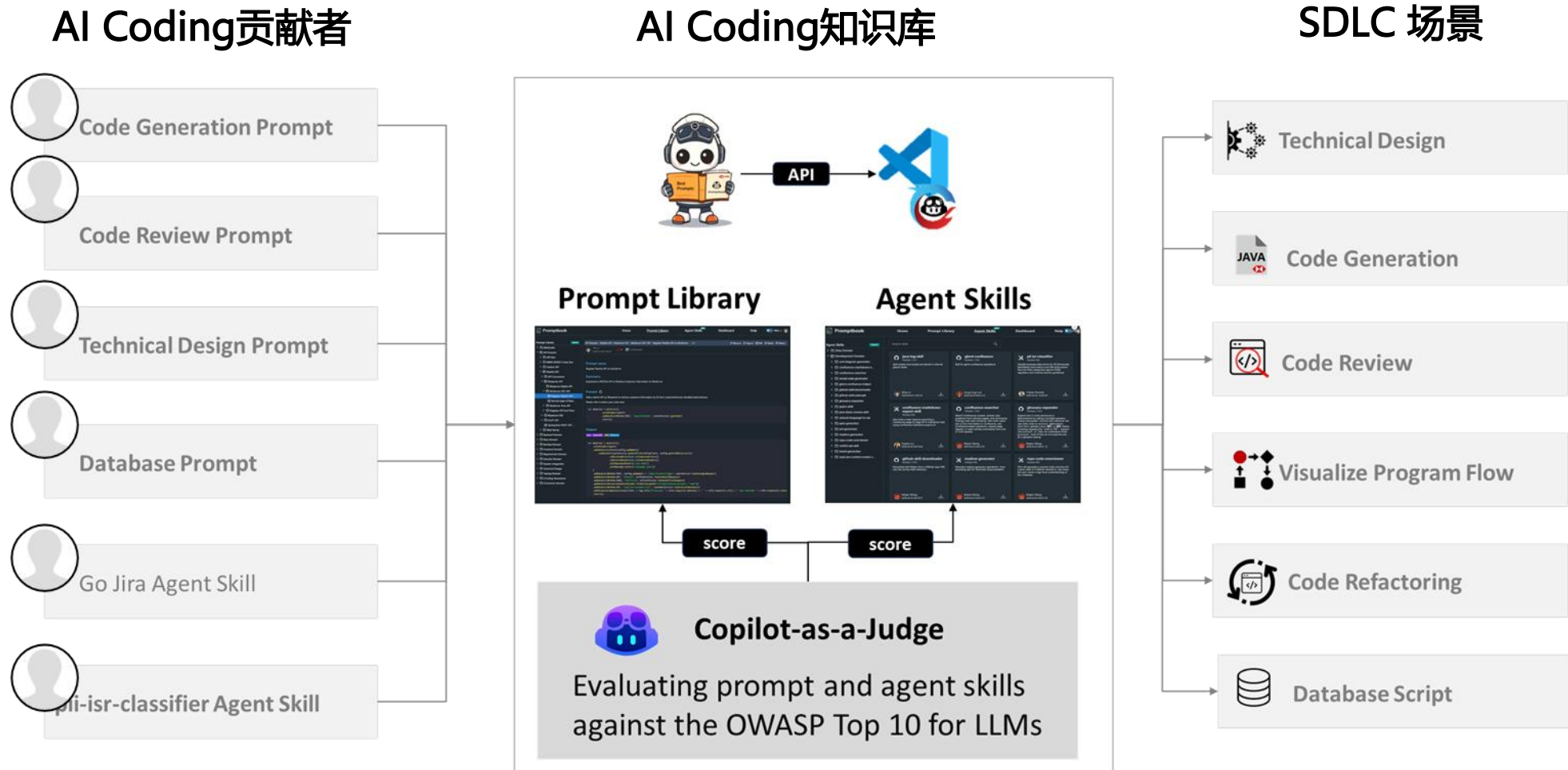


CODE REVIEW

#### 生成越快, Review 越重要

同时验证业务需求、内部技术规范、安全标准与 End-to-End Evidence

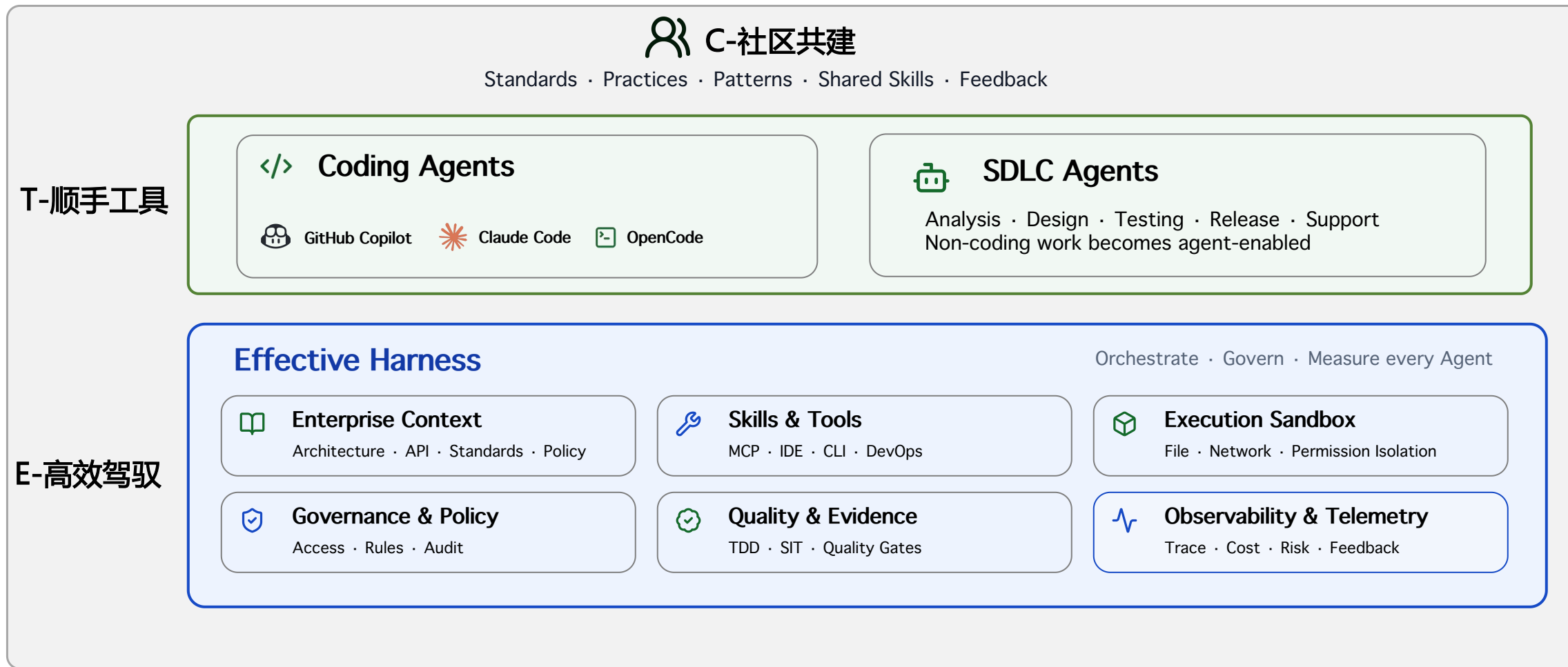
# 社区共建



04

# 回顾和总结

# ■ 如何解决规模化落地的五项挑战



# 三大核心启示 | Three Key Takeaways

01



竞争转向  
Engineering System

企业上下文、质量与治理才是长期壁垒。

02



洞察力 · 判断力 · 执行力  
仍是第一生产力

让 AI 放大智慧，而非替代责任。

03



衡量 Outcome  
而不是代码量与 Token

用效能、质量、风险和成本衡量结果。

洞察力 · 判断力 · 执行力 + Engineering System = Sustainable AI Productivity

# QCon

## 全球软件开发大会

上海  
站

# 大模型正在重新定义软件

Large Language Model Is Redefining The Software

2026 年 10 月 22-24 日 / 上海建工浦江皇冠假日酒店



参会  
咨询

**专题：从 App 到 Agent：字节跳动  
Supabase 的 Agent-Native 规模化实践**

专题演讲嘉宾：张国强

字节跳动 / 资深技术专家



**专题：复杂业务 Agent 的持续进化：  
快手电商导购的 Harness Loop 实践**

专题演讲嘉宾：包磊

快手 / 资深技术专家



**专题：从 Harness 到 Loop：阿福 Agent  
小队如何处理持续涌入的线上 Badcase**

专题演讲嘉宾：肖汉松

蚂蚁集团 / 阿福 Harness 架构组负责人



**专题：AI Coding 在大型客户端工程中的  
落地实践——从通才模型到专才 Agent**

专题演讲嘉宾：杨凯

小红书 / 客户端基础技术负责人



**专题：AI 时代的一人公司：从 Idea 到  
Growth 的完整 workflow**

专题演讲嘉宾：casa

独立开发



**专题：Agent 驱动的全平台 GUI 持续  
验证实践**

专题演讲嘉宾：王政达

微软 / SENIOR SOFTWARE ENGINEER



# THANKS

探索 AI 应用边界

Explore the limits of AI applications

## AiCon

全球人工智能开发与应用大会



wiky-李渭宁

Guangzhou, Guangdo...

